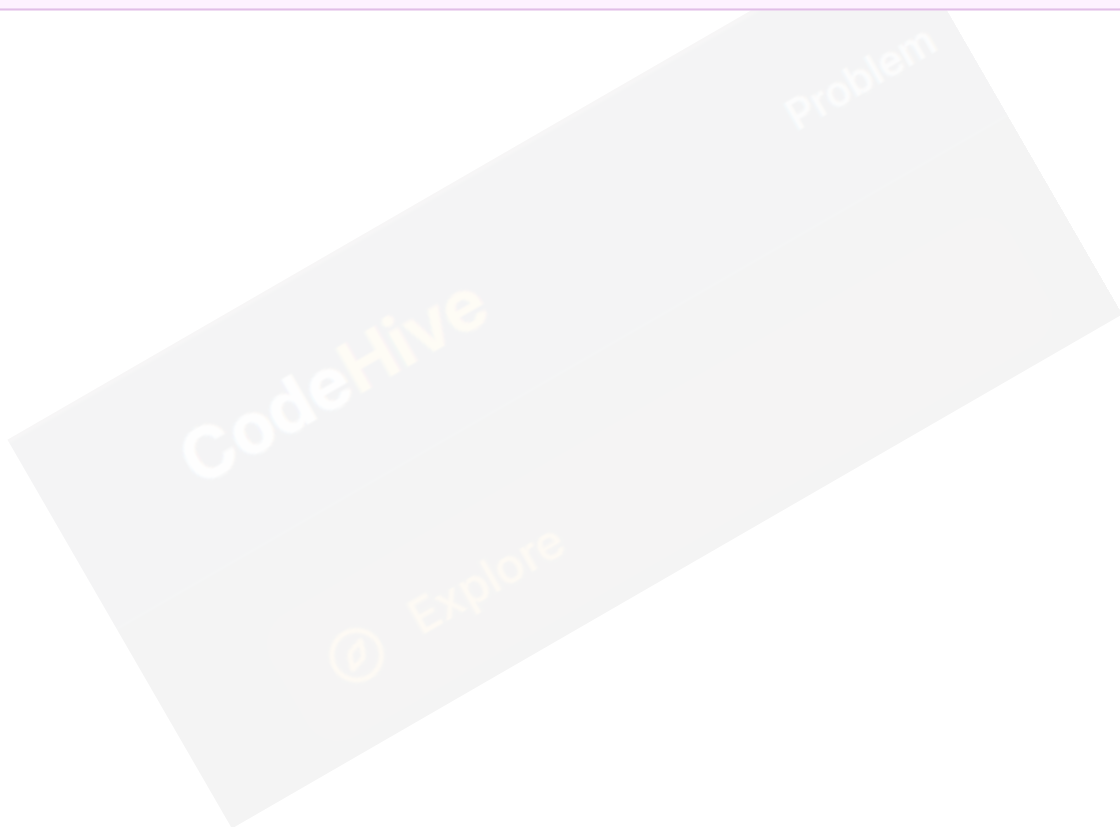


C STRUCTS CHALLENGE

A structs challenge in C usually tests how well you can define, initialize, access, and manipulate structures in a program. It is the bridge between simple variables and real-world data systems.

CodeHive Objective: Mastering structures is the first step toward database management and object-oriented thinking.



The Typical Task

In most challenges, you are required to:

- Define a structure with relevant fields (int, char, float).
- Take user input for these members.
- Perform a calculation (like average marks or salary tax).
- Display formatted output.



Sample Challenge

Problem Statement: Create a structure `Student` with `roll`, `name`, and `marks`.

Your program must read details of a specific student from the console and display the stored information back to the user.



Code Solution

```
#include <stdio.h>

struct Student {
    int roll;
    char name[20];
    float marks;
};

int main() {
    struct Student s;
    // ... logic continues ...
}
```

CodeHive

@ Explore

Taking Input

Using the dot operator (.) to populate the structure:

```
printf("Enter roll: ");  
scanf("%d", &s.roll);  
  
printf("Enter name: ");  
scanf("%s", s.name);  
  
printf("Enter marks: ");  
scanf("%f", &s.marks);
```

CodeHive

@ Explore

Formatted Output

Reporting the data back to the user:

```
printf("\nStudent Details:\n");  
printf("Roll: %d\n", s.roll);  
printf("Name: %s\n", s.name);  
printf("Marks: %.2f\n", s.marks);
```

CodeHive



Explore

Problem

Concepts Tested (Part 1)

Structure Definition: Can you correctly use the `struct` keyword and the closing semicolon?

Member Access: Using the `.` operator correctly for variables.



Concepts Tested (Part 2)

Real-world Data: Can you represent complex entities? For example, an Address structure containing City, Zip, and Street.



Challenge Level: Hard

Modify the logic to handle 5 students at once. This requires an **Array of Structures**:

```
struct Student class_list[5];  
for(int i=0; i<5; i++) {  
    scanf("%d", &class_list[i].roll);  
}
```

CodeHive

@ Explore

Problem

CodeHive Best Practices

- Use meaningful names for members.
- Keep structure definitions global if multiple functions need them.
- Initialize structures to zero if you want to avoid garbage values.



Exam Pitfalls

1. Forgetting the semicolon after `struct { ... };`.
2. Forgetting the `&` in `scanf` for integers/floats.
3. Trying to use `==` on structures (it doesn't work!).



Key Points (Very Important ★)

- Use the **struct** keyword for definition.
 - Access members with **dot (.) operator**.
 - Structures can store **heterogeneous** (different) data types.
 - Essential for competitive programming and practical exams.
-

Goal Reached: Mastered the Challenge!

CodeHive Learning Platform

CodeHive

@ Explore